

CMake Configuration for Key-Value Store

<https://gemini.google.com/app/ca0d7caf00642685>

User prompt: I am using this chat to continue my phase 1 of my key value store project. I have written the storage_engine_test code. The current project structure is: KeyValueType/ | | CMakeLists.txt | | proto/ | | src/ | | storage_engine.cpp | | storage_engine.h | | server.cpp | | client.cpp | | tests/ | | test_storage.cpp I updated the cmakefile.txt to create an executable for the storage_engine_test. Here is the updated cmakefile.txt file:

```
cmake_minimum_required(VERSION 3.8) project(KeyValueType CXX) # Force the C++17 standard required by gRPC 1.70+ set(CMAKE_CXX_STANDARD 17) set(CMAKE_CXX_STANDARD_REQUIRED ON) # 1. Locate the gRPC and Protobuf packages installed in your local directory find_package(gRPC CONFIG REQUIRED) find_package(Protobuf CONFIG REQUIRED) find_package(Threads REQUIRED) # Required for safe multi-threaded networking # Paths where the compiler will generate the stub files on the fly set(proto_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.cc") set(proto_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.h") set(grpc_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.cc") set(grpc_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.h") # 2. Automation: Tell CMake how to compile proto/kv.proto into C++ classes add_custom_command( OUTPUT "${proto_srcs}" "${proto_hdrs}" "${grpc_srcs}" "${grpc_hdrs}" COMMAND protoc --proto_path="${CMAKE_CURRENT_SOURCE_DIR}/proto" --cpp_out "${CMAKE_CURRENT_BINARY_DIR}" --grpc_out "${CMAKE_CURRENT_BINARY_DIR}" --plugin=protoc-gen-grpc=${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.grpc.pb.cc # Safe Generator Expression -I "${CMAKE_CURRENT_SOURCE_DIR}/proto" "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto" DEPENDS "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto" ) # Allow our files to find the auto-generated headers include_directories("${CMAKE_CURRENT_BINARY_DIR}" src) # 3. Compile the auto-generated Proto files into a single reusable library add_library(kv_grpc_proto ${grpc_srcs} ${proto_srcs}) target_link_libraries(kv_grpc_proto gRPC::grpc++ protobuf::libprotobuf) # 4. Define the core Storage Engine library add_library(storage_engine src/storage_engine.cpp src/storage_engine.h) # 5. Build the Server executable add_executable(kv_server src/server.cpp) target_link_libraries(kv_server storage_engine kv_grpc_proto Threads::Threads) # 6. Build the Client executable add_executable(kv_client src/client.cpp) target_link_libraries(kv_client kv_grpc_proto Threads::Threads) # 7. Build the Storage Engine test executable add_executable(test_storage tests/storage_engine_test.cpp) target_link_libraries(test_storage PRIVATE storage_engine Threads::Threads) target_compile_options(test_storage PRIVATE -fsanitize=thread -g -O1) target_link_options(test_storage PRIVATE -fsanitize=thread) Now I created ./build, and then ran "cmake .." in terminal while in that build folder. But I got this error: build % cmake .. CMake Deprecation Warning at CMakeLists.txt:1 (cmake_minimum_required): Compatibility with CMake < 3.10 will be removed from a future version of CMake. Update the VERSION argument <min> value. Or, use the <min>...<max> syntax to tell CMake that the project requires at least <min> but has been updated to work with policies introduced by <max> or earlier. CMake Error at CMakeLists.txt:9 (find_package): Found package configuration file: /Users/subhamoneeshkasireddi/.local/lib/cmake/grpc/grpcConfig.cmake but it set gRPC_FOUND to FALSE so package "gRPC" is considered to be NOT FOUND. Reason given by package: The following imported targets are referenced, but are missing: protobuf::libprotobuf protobuf::libprotoc -- Configuring incomplete, errors occurred! Please help
```

Response: The error and warning are happening due to two specific issues in your CMakeLists.txt configuration:

1. Root Cause of the Error (gRPC_FOUND set to FALSE)

In your CMakeLists.txt, you wrote:

```
find_package(gRPC CONFIG REQUIRED)
find_package(Protobuf CONFIG REQUIRED)
```

gRPC directly depends on Protobuf. When CMake loads grpcConfig.cmake, gRPC checks whether the imported CMake targets protobuf::libprotobuf and protobuf::libprotoc already exist in the build context. Because find_package(Protobuf ...) was placed after gRPC, those targets did not exist yet, causing gRPC to report that required targets were missing.

Fix: find_package(Protobuf CONFIG REQUIRED) must always be called before find_package(gRPC CONFIG REQUIRED).

2. Cause of the Deprecation Warning

`cmake_minimum_required(VERSION 3.8)` uses a CMake policy version that modern CMake versions (3.20+) flag as deprecated.

Fix: Update `cmake_minimum_required(VERSION 3.8)` to `cmake_minimum_required(VERSION 3.16)`.

3. File Path Mismatch (Bonus Check)

In your project structure, your test file is named `tests/test_storage.cpp`. However, inside section 7 of your `CMakeLists.txt`, you listed `tests/storage_engine_test.cpp`. If uncorrected, `make` would throw a "No such file or directory" error when compiling the test executable.

Updated `CMakeLists.txt`

Here is the corrected `CMakeLists.txt` file with all fixes applied:

```
cmake_minimum_required(VERSION 3.16)
project(KeyValueStore CXX)

# Force the C++17 standard required by gRPC
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# 1. Locate Protobuf FIRST, then gRPC
find_package(Protobuf CONFIG REQUIRED)
find_package(gRPC CONFIG REQUIRED)
find_package(Threads REQUIRED) # Required for safe multi-threaded networking

# Paths where the compiler will generate the stub files on the fly
set(proto_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.cc")
set(proto_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.pb.h")
set(grpc_srcs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.cc")
set(grpc_hdrs "${CMAKE_CURRENT_BINARY_DIR}/kv.grpc.pb.h")

# 2. Automation: Tell CMake how to compile proto/kv.proto into C++ classes
add_custom_command(
    OUTPUT "${proto_srcs}" "${proto_hdrs}" "${grpc_srcs}" "${grpc_hdrs}"
    COMMAND protobuf::protoc
    ARGS --cpp_out "${CMAKE_CURRENT_BINARY_DIR}"
         --grpc_out "${CMAKE_CURRENT_BINARY_DIR}"
         --plugin=protoc-gen-grpc=${<TARGET_FILE:gRPC::grpc_cpp_plugin>}
         -I "${CMAKE_CURRENT_SOURCE_DIR}/proto"
         "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto"
    DEPENDS "${CMAKE_CURRENT_SOURCE_DIR}/proto/kv.proto"
)

# Allow our files to find the auto-generated headers
include_directories("${CMAKE_CURRENT_BINARY_DIR}" src)

# 3. Compile the auto-generated Proto files into a single reusable library
add_library(kv_grpc_proto ${grpc_srcs} ${proto_srcs})
target_link_libraries(kv_grpc_proto gRPC::grpc++ protobuf::libprotobuf)

# 4. Define the core Storage Engine library
add_library(storage_engine src/storage_engine.cpp src/storage_engine.h)

# 5. Build the Server executable
add_executable(kv_server src/server.cpp)
target_link_libraries(kv_server storage_engine kv_grpc_proto Threads::Threads)

# 6. Build the Client executable
add_executable(kv_client src/client.cpp)
target_link_libraries(kv_client kv_grpc_proto Threads::Threads)
```

```
# 7. Build the Storage Engine test executable
add_executable(test_storage
    tests/test_storage.cpp
)

target_link_libraries(test_storage
    PRIVATE
        storage_engine
        Threads::Threads
)

target_compile_options(test_storage PRIVATE
    -fsanitize=thread
    -g
    -O1
)

target_link_options(test_storage PRIVATE
    -fsanitize=thread
)
```

How to Rebuild

Because CMake caches failed search results inside the `build/` directory, clear out old build artifacts before re-running CMake:

```
cd build
rm -rf *
cmake -DCMAKE_PREFIX_PATH=$HOME/.local ..
make -j$(sysctl -n hw.ncpu)
```

Passing `-DCMAKE_PREFIX_PATH=$HOME/.local` tells CMake exactly where your locally installed gRPC and Protobuf libraries are located. You can then execute `./test_storage` to run your test suite under ThreadSanitizer!

User prompt: thanks, swapping worked.

Response: Glad to hear that fixed the issue!

Now that your `CMakeLists.txt` is configured and CMake generates the build files cleanly, here are the immediate next steps to complete Phase 1:

Step 1: Run your Storage Engine Tests

First, compile and run your unit test executable to ensure your core `StorageEngine` is working and completely free of thread-safety data races:

```
# Inside your KeyValueStore/build directory:
make test_storage
./tests/test_storage
```

Since ThreadSanitizer (`-fsanitize=thread`) is enabled for `test_storage`, it will validate both your functional logic and monitor all concurrent reads/writes across threads.

Step 2: What's Next?

Once your tests pass cleanly with `[✓] Functional validation passed successfully.`, you're ready to bridge your storage engine to the network layer:

1. **Implement `src/server.cpp`:** Inherit from the generated gRPC base class

(kvstore::KeyValueStore::Service) and wire up the Put, Get, and Delete RPC calls to forward requests to your StorageEngine.

2. **Implement src/client.cpp**: Create a CLI client that connects to localhost:50051 using gRPC stubs to test network operations.

Let me know if your test run passes or if you'd like to start implementing src/server.cpp next!

User prompt: Let's get to the step 4 of this phase 1, that is, to Implement the gRPC server, calling into StorageEngine. Explain each and every line of this code(src/server.cpp) like I am a beginner to cpp programming, knowing just basics like loops, but no knowledge of many libraries or other non basic concepts, and with not knowledge of grpc or protobuf: #include <grpcpp/grpcpp.h> #include "kv.grpc.pb.h" #include "storage_engine.h" class KVServiceImpl final : public kvstore::KeyValueStore::Service { public: grpc::Status Put(grpc::ServerContext* context, const kvstore::PutRequest* request, kvstore::PutResponse* response) override { storage_.Put(request->key(), request->value()); response->set_success(true); return grpc::Status::OK; } grpc::Status Get(grpc::ServerContext* context, const kvstore::GetRequest* request, kvstore::GetResponse* response) override { auto val = storage_.Get(request->key()); if (val.has_value()) { response->set_found(true); response->set_value(*val); } else { response->set_found(false); } return grpc::Status::OK; } grpc::Status Delete(grpc::ServerContext* context, const kvstore::DeleteRequest* request, kvstore::DeleteResponse* response) override { bool ok = storage_.Delete(request->key()); response->set_success(ok); return grpc::Status::OK; } private: StorageEngine storage_; }; void RunServer(const std::string& address) { KVServiceImpl service; grpc::ServerBuilder builder; builder.AddListeningPort(address, grpc::InsecureServerCredentials()); builder.RegisterService(&service); auto server = builder.BuildAndStart(); std::cout << "Server listening on " << address << std::endl; server->Wait(); } int main(int argc, char** argv) { std::string address = "0.0.0.0:50051"; if (argc > 1) address = argv[1]; // allow port override for multi-node later RunServer(address); return 0; }

Response: Before diving into the code line-by-line, let's build a clear mental picture of what src/server.cpp actually does.

The Big Picture Mental Model

Think of your project as a **restaurant**:

1. **The Client (client.cpp)**: The customer ordering food over the phone.
2. **gRPC & Protobuf (kv.proto)**: The phone line and standard menu format. It guarantees that requests and responses are packaged cleanly.
3. **The Server (server.cpp)**: The waiter at the restaurant. The waiter receives phone orders, walks over to the kitchen, gets or updates food items, and replies back on the phone.
4. **The Storage Engine (storage_engine.h/cpp)**: The kitchen/pantry where data is actually stored in memory (std::unordered_map).

src/server.cpp acts as the **bridge/translator** between the network world (gRPC requests coming over the internet) and your C++ internal storage engine (StorageEngine).

Detailed Line-by-Line Breakdown

1. The Header Includes

```
#include <grpcpp/grpcpp.h>
#include "kv.grpc.pb.h"
#include "storage_engine.h"
```

- **#include <grpcpp/grpcpp.h>**: Imports the main C++ library for gRPC. It provides networking tools to build network servers, listen on ports, and manage connections.
- **#include "kv.grpc.pb.h"**: Imports the header file **automatically generated by CMake/Protobuf**

from your `kv.proto` file. It contains the C++ code for `kvstore::KeyValueStore::Service`, `PutRequest`, `GetResponse`, etc.

- `#include "storage_engine.h"`: Imports your custom `StorageEngine` class header, allowing the server to interact with your thread-safe key-value database.

2. Class Definition and Inheritance

```
class KVServiceImpl final : public kvstore::KeyValueStore::Service {
```

- **class KVServiceImpl**: We define a custom class named `KVServiceImpl` (short for *Key-Value Service Implementation*).
- **final**: A C++ keyword meaning “no other class is allowed to inherit from this class.” It helps the C++ compiler optimize code execution.
- **public kvstore::KeyValueStore::Service**: This is **Class Inheritance**.
 - `kvstore::KeyValueStore::Service` is an automatically generated base class from `kv.proto`.
 - By inheriting from it, our `KVServiceImpl` class promises to handle the `Put`, `Get`, and `Delete` network calls defined in `kv.proto`.
- **:: (Scope Resolution Operator)**: In C++, double colons `::` work like opening nested subfolders:
 - `kvstore` = top-level namespace.
 - `KeyValueStore` = class inside that namespace.
 - `Service` = class inside `KeyValueStore` generated by gRPC.

3. The `Put` Method (Handling Put Requests)

```
public:
    grpc::Status Put(grpc::ServerContext* context,
                    const kvstore::PutRequest* request,
                    kvstore::PutResponse* response) override {
        storage_.Put(request->key(), request->value());
        response->set_success(true);
        return grpc::Status::OK;
    }
```

- **public::** Makes the functions below accessible to the gRPC framework.
- **grpc::Status**: The return type of the function. It tells gRPC whether the network operation succeeded or failed (e.g., network error, client disconnected, or OK).
- **grpc::ServerContext* context**: A pointer to gRPC metadata about this specific connection (such as client IP address, request timeout, or security credentials).
- **const kvstore::PutRequest* request**:
 - `request` is a **pointer** (memory reference `*`) pointing to the message sent by the client.
 - `const` ensures we **cannot accidentally modify** the client's request message.
 - `kvstore::PutRequest` contains fields defined in `kv.proto` (`key` and `value`).
- **kvstore::PutResponse* response**: A pointer to an empty response object created by gRPC. Our job is to write the answer into this object.
- **override**: A safety keyword that tells the C++ compiler: “Make sure this function matches the exact name and parameters defined in the parent `Service` class.”

- `storage_.Put(request->key(), request->value());`
 - `request->key()`: Call Protobuf's auto-generated getter method to retrieve the string key sent by the client.
 - `request->value()`: Retrieve the string/bytes value sent by the client.
 - `->`: The **arrow operator** is used in C++ to call a function on an object through a pointer (`request` is a pointer).
 - `storage_.Put(...)`: Passes the key and value to our internal `StorageEngine` instance to store it in memory.
- `response->set_success(true);`: Calls Protobuf's auto-generated setter method to record `success = true` in the reply message.
- `return grpc::Status::OK;`: Returns a status code telling gRPC: *"The operation completed without errors; send the response back across the network to the client."*

4. The `Get` Method (Handling Fetch Requests)

```
grpc::Status Get(grpc::ServerContext* context,
                 const kvstore::GetRequest* request,
                 kvstore::GetResponse* response) override {
    auto val = storage_.Get(request->key());
    if (val.has_value()) {
        response->set_found(true);
        response->set_value(*val);
    } else {
        response->set_found(false);
    }
    return grpc::Status::OK;
}
```

- `auto val = storage_.Get(request->key());`
 - Calls our `StorageEngine::Get` method using the key requested by the client.
 - `auto`: Automatic type deduction in C++. Here, `val` is inferred to be `std::optional<std::string>`.
- `if (val.has_value())`:
 - `std::optional` holds either a string value or nothing (`std::nullopt`). `.has_value()` returns `true` if the key was found in our database.
- `response->set_found(true);`: Tells the client the key exists.
- `response->set_value(*val);`
 - `*val`: The **dereference operator** `*` extracts the actual `std::string` stored inside the `std::optional` container.
 - `.set_value(...)`: Writes that string into the response message.
- `else { response->set_found(false); [cite_start]}`: If the key didn't exist, we inform the client by setting `found = false`.
- `return grpc::Status::OK;`: Returns success to the gRPC runtime layer.

5. The `Delete` Method (Handling Removal Requests)

```
grpc::Status Delete(grpc::ServerContext* context,
                    const kvstore::DeleteRequest* request,
                    kvstore::DeleteResponse* response) override {
```

```

        bool ok = storage_.Delete(request->key());
        response->set_success(ok);
        return grpc::Status::OK;
    }

```

- `bool ok = storage_.Delete(request->key());`: Attempts to remove the key from `StorageEngine`. Returns `true` if it existed and was deleted, or `false` if the key was missing.
- `response->set_success(ok);`: Populates the response boolean field with `ok`.
- `return grpc::Status::OK;`: Completes the gRPC handler routine.

6. Private Member Variable

```

private:
    StorageEngine storage_;
};

```

- `private::` Variables placed here cannot be accessed from outside this class.
- `StorageEngine storage_;`: Creates **one single instance** of your `StorageEngine` class.
 - Every incoming network thread calling `Put`, `Get`, or `Delete` shares this exact same `storage_` object.
 - Because your `StorageEngine` uses `std::shared_mutex` inside, all concurrent gRPC thread accesses remain completely thread-safe.

7. The `RunServer` Helper Function

```

void RunServer(const std::string& address) {
    KVServiceImpl service;
    grpc::ServerBuilder builder;
    builder.AddListeningPort(address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);
    auto server = builder.BuildAndStart();
    std::cout << "Server listening on " << address << std::endl;
    server->Wait();
}

```

- `void RunServer(const std::string& address)`: A helper function that configures and launches the network listener on a specific address (e.g., `"0.0.0.0:50051"`).
- `KVServiceImpl service;`: Instantiates our service handler class.
- `grpc::ServerBuilder builder;`: Uses the **Builder Design Pattern** provided by gRPC to set up server options step-by-step before starting.
- `builder.AddListeningPort(address, grpc::InsecureServerCredentials());`:
 - Tells gRPC which network address and port to bind to.
 - `grpc::InsecureServerCredentials()`: Configures plain-text (unencrypted) TCP communication for fast, local development testing.
- `builder.RegisterService(&service);`: Registers our `service` object memory address (`&service`) with the builder so gRPC routes incoming RPC requests to our `Put`, `Get`, and `Delete` methods.
- `auto server = builder.BuildAndStart();`: Compiles the settings, binds to the TCP port, and spins up a thread pool listening for incoming client connections.
- `std::cout << ...`: Prints a confirmation log to terminal console.
- `server->Wait();`: **Blocks the main thread indefinitely.** This stops the program from exiting

immediately, keeping the server listening until you manually stop it with `Ctrl + C`.

8. The `main` Entry Point

```
int main(int argc, char** argv) {
    std::string address = "0.0.0.0:50051";
    if (argc > 1) address = argv[1]; // allow port override for multi-node later
    RunServer(address);
    return 0;
}
```

- `int main(int argc, char** argv)`: The standard entry point of any C++ program:
 - `argc` (*argument count*): Number of command line arguments provided.
 - `argv` (*argument vector*): Array of strings containing those arguments.
- `std::string address = "0.0.0.0:50051";`: Default listening IP (`0.0.0.0` listens on all network interfaces on your computer) and port `50051`.
- `if (argc > 1) address = argv[1];`: If you run `./kv_server 0.0.0.0:50052` from terminal, it overrides the default address with your argument. (*This will be critical when setting up a 6-node cluster on different ports later!*)
- `RunServer(address)`: Calls our helper function to spin up the server.
- `return 0;`: Standard exit code indicating success.

Summary Checklist for a Beginner

C++ / gRPC Syntax	What it Means in Plain English
<code>namespace::Class</code>	Drilling down into nested namespaces/classes.
<code>* in parameters (PutRequest*)</code>	Pass-by-pointer (referencing memory directly without creating heavy copies).
<code>-></code> Arrow operator	Accessing functions on a pointer object (e.g. <code>request->key()</code>).
<code>override</code>	Safety check ensuring we properly implement gRPC parent class functions.
<code>grpc::Status::OK</code>	Telling gRPC "Network call finished without issues".
<code>server->Wait()</code>	Keeping the process alive permanently in the background to handle incoming requests.

User prompt: will this line work in my pc: `#include <grpcpp/grpcpp.h>`, like how and from where will it get included from?

Response: **Yes, it will work seamlessly!** Here is exactly how and from where your compiler locates `#include <grpcpp/grpcpp.h>`.

1. Where is it physically on your PC?

When you built and installed gRPC earlier, gRPC placed its C++ library header files into your local user installation folder. On macOS/Linux, it lives here:

`~/ .local/include/grpcpp/grpcpp.h`

(Or in `/usr/local/include/grpcpp/grpcpp.h` if installed system-wide).

If you open your terminal right now and run this command, you can see the file directly on your disk:

```
ls ~/.local/include/grpcpp/grpcpp.h
```

2. How does C++ know where to look?

In C++, there is a distinct difference between using quotes ("...") and angle brackets (<...>):

- **Quotes** `"storage_engine.h"`: Tells the preprocessor to look in the **current working directory** first.
- **Angle Brackets** `<grpcpp/grpcpp.h>`: Tells the preprocessor to skip the local project folder and search in **Include Search Paths** provided to the compiler.

3. How CMake bridges the gap

You might wonder: *How does Clang/GCC know to search inside ~/.local/include?*

This is where **CMake** does the heavy lifting behind the scenes:

1. `find_package(gRPC CONFIG REQUIRED)` inside your `CMakeLists.txt` searches your system and finds where gRPC was installed.
2. When CMake builds your server executable, it automatically passes an include flag (`-I`) to your compiler:


```
clang++ -I/Users/yourname/.local/include -c src/server.cpp
```
3. When the compiler encounters `#include <grpcpp/grpcpp.h>`, it appends `grpcpp/grpcpp.h` to `-I/Users/yourname/.local/include`, resulting in `/Users/yourname/.local/include/grpcpp/grpcpp.h`.
4. It finds the file, opens it, and imports all gRPC network tools into `src/server.cpp`!

Summary Flow



User prompt: like right now my vscode is showing red underline under that line, saying: #include errors detected. Please update your includePath. Squiggles are disabled for this translation unit

Response: Don't worry—**your code will still compile fine!** This red underline is purely a visual issue in VS Code. The compiler (Clang/GCC) uses CMake to find gRPC, but VS Code's C/C++ editor extension (IntelliSense) operates separately and doesn't automatically look inside `~/.local/include` unless you tell it to.

Here are two quick ways to fix the red squiggles.

Method 1: Let CMake Handle It Automatically (Recommended)

Since you already have `CMakeLists.txt` configured, the easiest fix is to tell VS Code to read your include paths directly from CMake.

1. Open the Command Palette in VS Code:
 - **macOS:** `Cmd + Shift + P`
 - **Linux/Windows:** `Ctrl + Shift + P`
2. Type and select: **C/C++: Change Configuration Provider...**
3. Select **CMake Tools**.
4. Open the Command Palette again and run: **CMake: Configure**.

VS Code will now parse your CMake configuration, locate gRPC, and clear the red squiggles automatically.

Method 2: Manually Update `includePath` in VS Code

If you prefer to configure IntelliSense manually without the CMake extension:

1. Open the Command Palette (`Cmd + Shift + P` / `Ctrl + Shift + P`).
2. Type and select: **C/C++: Edit Configurations (JSON)**.
3. This creates or opens a file at `.vscode/c_cpp_properties.json`.
4. Add `"${userHome}/.local/include/**"` and `"${workspaceFolder}/build/**"` to your `includePath` array:

```
{
  "configurations": [
    {
      "name": "Mac / Linux",
      "includePath": [
        "${workspaceFolder}/**",
        "${userHome}/.local/include/**",
        "${workspaceFolder}/build/**"
      ],
      "cStandard": "c11",
      "cppStandard": "c++17"
    }
  ],
  "version": 4
}
```

Why add `${workspaceFolder}/build/**`? This ensures VS Code can also find the generated Protobuf headers (`kv.pb.h` and `kv.grpc.pb.h`) that CMake creates in your build folder when compiling!